

TP2 – Infrastructure DevOps & Automatisation

TP2 – Infrastructure DevOps & Automatisation avec Ansible et Terraform

- Présentation

- Documentation

- Structure du dépôt

- Architecture

- Exécution en local

- Prérequis

01 – Vue d'ensemble de l'infrastructure

- Architecture

- Composants

- Groupe d'hôtes (inventaire)

- Mise à l'échelle

02 – Configuration du Runner Auto-Hébergé GitHub Actions

- Pourquoi un runner auto-hébergé ?

- Étapes d'enregistrement

- Utilisation dans le pipeline

- Prérequis sur la machine du runner

03 – Terraform & Génération Dynamique de l'Inventaire

- Objectif

- Fichiers

- Fonctionnement

- Variables

- Exécution en local

04 – Playbook Ansible & Rôles

- Structure du playbook

- Plays dans `site.yml`

- Notes de conception

- Exécution en local

05 – Répartition de Charge avec nginx

- Vue d'ensemble

- Algorithme

- Configuration upstream générée (`lb.conf.j2`)

- Ajouter un nouveau serveur web

- Vérification de santé

- Tester la distribution de charge

06 – Déroulement du Pipeline CI/CD

- Déclencheur

- Vue d'ensemble du pipeline

- Décisions clés du pipeline

- Résolution des problèmes

- Preuve d'exécution sur le runner local

07 – Tentatives de Provisionnement de la VM Windows

Objectif

Contexte matériel

Tentative 1 — ISO Windows Server 2022 x86 via UTM (émulation)

Tentative 2 — ISO Windows Server ARM64 native via uupdump.net

Tentative 3 — Démarrage de l'ISO ARM64 dans UTM

Tentative 4 — VM Azure Cloud (Windows Server 2022 Datacenter)

Conclusion

Ce qu'aurait fait l'automatisation Windows

TP2 – Infrastructure DevOps & Automatisation avec Ansible et Terraform

École : EFREI | **Promotion :** M1-DEV1 2026 | **Cours :** DevOps – Culture, Pratiques et Outils

Professeur : Issiaka KONE

Présentation

Ce projet automatise le déploiement d'un **serveur web nginx** servant une page HTML statique, via un pipeline entièrement automatisé :

GitHub Actions (runner auto-hébergé)

└─ Job 1 – Terraform → génère ansible/inventory.ini depuis un template Jinja2-like

└─ Job 2 – Docker → construit et démarre le conteneur Ubuntu cible

└─ Job 3 – Ansible → installe nginx & déploie la page HTML sur les hôtes cibles

Documentation

La documentation complète est disponible dans le dossier [docs/](#) :

Section	Fichier
Vue d'ensemble de l'architecture	docs/01-infrastructure.md
Runner auto-hébergé	docs/02-runner-setup.md
Terraform & inventaire dynamique	docs/03-terraform.md
Playbook Ansible & rôles	docs/04-ansible.md
Load balancer nginx	docs/05-load-balancing.md
Pipeline CI/CD	docs/06-pipeline.md

Section	Fichier
Tentatives VM Windows	docs/07-windows-vm-attempts.md

Structure du dépôt

```

TP2/
├── .github/workflows/deploy.yml # Pipeline GitHub Actions (3
jobs)
├── terraform/
│   ├── main.tf                # Config Terraform – génère
l'inventaire via templatefile()
│   └── inventory.tpl          # Template Jinja2-like pour
l'inventaire Ansible
├── docker/
│   ├── docker-compose.yml    # Conteneur ubuntu-target (SSH +
nginx)
│   └── gen-keys.sh            # Génère la paire de clés SSH
ED25519
│   └── ubuntu-target/        # Dockerfile + authorized_keys
├── ansible/
│   └── ansible.cfg            # Configuration Ansible du
projet
│   ├── site.yml              # Playbook principal (5 plays)
│   └── roles/
│       └── nginx/            # Installe nginx + déploie la
page HTML
│           ├── timezone/    # Gestion du fuseau horaire
(Linux, compatible Docker)
│           └── loadbalancer/ # Reverse proxy nginx sur le
port 8081
│               └── windows_webserver/ # Rôle Windows (stub documenté)
├── docs/                      # Documentation complète du TP
└── screenshots/               # Captures d'écran du pipeline

```

Architecture

Composant	Technologie	Rôle
CI/CD	GitHub Actions	Orchestre les jobs du pipeline
Runner	Runner auto-hébergé (local)	Exécuteur exclusif du pipeline
IaC	Terraform + templatefile()	Génère ansible/inventory.ini
Gestion de config	Ansible	Installe nginx, déploie le HTML, gère le fuseau horaire

Composant	Technologie	Rôle
Serveur web	nginx (Docker)	Sert la page HTML statique (port 8080)
Load balancer	nginx (Docker)	Distribue le trafic (port 8081)

Exécution en local

1. Générer l'inventaire avec Terraform

```
cd terraform
terraform init
terraform apply -auto-approve
```

2. Démarrer le conteneur cible

```
cd docker
bash gen-keys.sh
docker compose up -d --build
```

3. Lancer le playbook Ansible

```
cd ansible
ansible-playbook -i inventory.ini site.yml
```

4. Vérifier

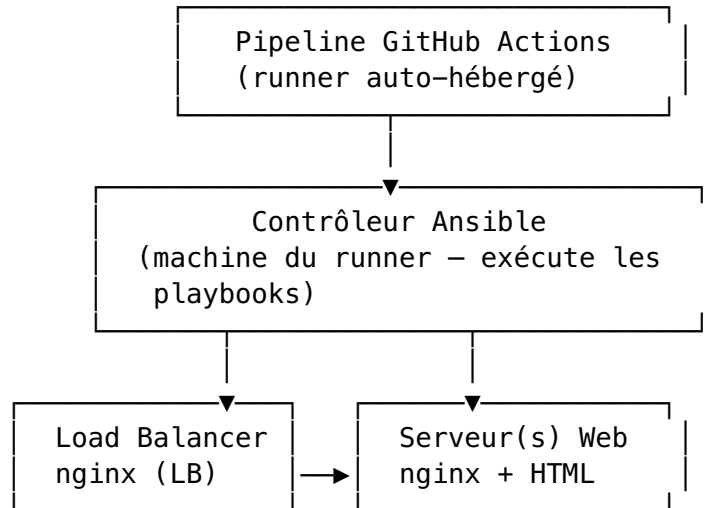
```
curl http://127.0.0.1:8080 # page nginx
curl http://127.0.0.1:8081/health # load balancer → OK
```

Prérequis

- Runner GitHub Actions auto-hébergé enregistré sur ce dépôt
- Terraform ≥ 1.0 installé sur la machine du runner
- Ansible ≥ 2.12 installé sur la machine du runner
- Docker + Docker Compose installés sur la machine du runner

01 – Vue d’ensemble de l’infrastructure

Architecture



Composants

Composant	Rôle
GitHub Actions	Orchestration CI/CD
Runner auto-hébergé	Exécute les jobs du pipeline en local
Terraform	Génère <code>ansible/inventory.ini</code> depuis un template
Contrôleur Ansible	Lance les playbooks sur les hôtes cibles
nginx (webservers)	Sert la page HTML statique
nginx (loadbalancer)	Distribue le trafic entre les serveurs web

Groupes d’hôtes (inventaire)

Groupe	IP par défaut	Rôle
webservers	127.0.0.1	Cibles de déploiement nginx + HTML
loadbalancer	127.0.0.1	Reverse proxy nginx / upstream

Mise à l’échelle

Pour ajouter des serveurs web, renseigner la variable GitHub Actions `WEB_HOSTS` avec un tableau JSON :

```
["192.168.1.10", "192.168.1.11", "192.168.1.12"]
```

Terraform régénère l'inventaire et Ansible reconfigure automatiquement les serveurs web ainsi que le bloc upstream du load balancer.

02 – Configuration du Runner Auto-Hébergé GitHub Actions

Pourquoi un runner auto-hébergé ?

Un runner auto-hébergé garantit que le pipeline s'exécute sur **votre propre machine** (ou VM), ce qui permet de : - Contrôler l'environnement (outils, réseau, accès sudo)
- Prouver que le pipeline ne tourne pas sur l'infrastructure partagée de GitHub

Étapes d'enregistrement

1. Accéder aux paramètres du runner

<https://github.com/LT0ssian/automation-tp2> → **Settings** → **Actions** → **Runners** → **New self-hosted runner**

2. Téléchargement et configuration

```
mkdir ~/actions-runner && cd ~/actions-runner
# Télécharger (utiliser l'URL exacte affichée sur GitHub – inclut l'OS/architecture)
curl -o actions-runner.tar.gz -L <URL_DEPUIS_GITHUB>
tar xzf actions-runner.tar.gz

# Enregistrer (le token est affiché sur GitHub – valide 1h)
./config.sh \
  --url https://github.com/LT0ssian/automation-tp2 \
  --token <TOKEN_DEPUIS_GITHUB>
```

3. Démarrer le runner

```
# Temporaire / test
./run.sh

# Persistant – service systemd (recommandé)
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

4. Vérification

Settings → **Actions** → **Runners** → le runner apparaît avec le statut **Idle** (point vert).

Utilisation dans le pipeline

Dans `.github/workflows/deploy.yml`, chaque job déclare :

```
runs-on: self-hosted
```

Cela garantit que GitHub ne basculera jamais sur un runner partagé hébergé.

Prérequis sur la machine du runner

Outil	Commande d'installation
Ansible	<code>sudo apt install ansible</code>
Terraform	Voir terraform.io/install
sudo	Requis pour <code>become: true</code> dans les playbooks

03 – Terraform & Génération Dynamique de l'Inventaire

Objectif

Terraform remplace l'`inventory.ini` statique par un fichier **généré dynamiquement** à chaque exécution du pipeline, grâce à un template Jinja2-like (`inventory.tpl`).

Fichiers

Fichier	Rôle
<code>terraform/main.tf</code>	Déclare les variables et la ressource <code>local_file</code>
<code>terraform/inventory.tpl</code>	Template rendu en ansible/ <code>inventory.ini</code>
<code>ansible/inventory.ini</code>	Fichier de sortie généré — ne jamais versionner ce fichier

Fonctionnement

main.tf utilise la fonction native Terraform `templatefile()` :

```
resource "local_file" "ansible_inventory" {
  content = templatefile("${path.module}/inventory.tpl", {
    web_hosts = var.web_hosts    # liste des IPs des serveurs web
    lb_host   = var.lb_host      # IP du load balancer
    user      = var.ansible_user
    connection = var.ansible_connection
  })
  filename = "${path.module}/../ansible/inventory.ini"
}
```

Le template (`inventory.tpl`) itère sur la liste `web_hosts` :

```
[webservers]
%{ for idx, h in web_hosts ~}
${h} ansible_user=${user} ansible_connection=${connection}
server_id=${idx + 1}
%{ endfor ~}

[loadbalancer]
${lb_host} ansible_user=${user} ansible_connection=${connection}
```

Variables

Variable	Valeur par défaut	Surcharge via
web_hosts	["127.0.0.1"]	Variable GitHub Actions WEB_HOSTS (tableau JSON)
lb_host	127.0.0.1	Variable GitHub Actions LB_HOST
ansible_user	ubuntu	Variable GitHub Actions ANSIBLE_USER
ansible_connection	local	Variable GitHub Actions ANSIBLE_CONNECTION

Exécution en local

```
cd terraform
terraform init
terraform apply -auto-approve
cat ../ansible/inventory.ini    # inspecter le fichier généré
```

04 – Playbook Ansible & Rôles

Structure du playbook

```
ansible/
├─ ansible.cfg
├─ site.yml                # point d'entrée – cinq plays
├─ roles/
│   └─ nginx/              # installe nginx + déploie la page
HTML
│   └─ timezone/          # gestion du fuseau horaire
(Linux)
│   └─ loadbalancer/      # reverse proxy nginx sur le port
8081
└─ windows_webserver/     # stub Windows (voir limitations)
```

Plays dans site.yml

Play 1 – Collecte de faits sur le contrôleur (localhost)

Affiche l'OS, le hostname et l'IPv4 du contrôleur Ansible.

Play 2 – Serveurs web nginx

Installe nginx et déploie la page HTML personnalisée sur tous les hôtes [webservers]. nginx sert la page sur le **port 80**.

Play 3 – Gestion du fuseau horaire (Linux)

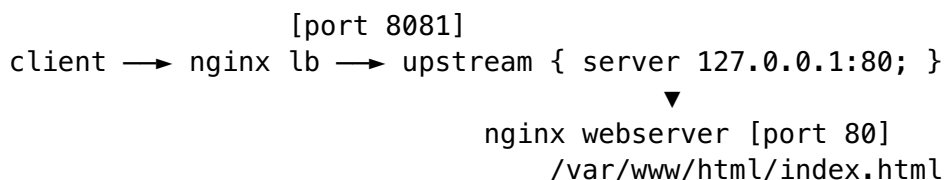
```
- name: Gestion du fuseau horaire sur les nœuds Linux
  hosts: webservers
  become: true
  roles:
    - timezone
```

Le rôle timezone utilise la manipulation directe des fichiers /etc/timezone + /etc/localtime (sans dépendance à hwclock ni timedatectl, non disponibles dans les conteneurs Docker) :

1. Écrit Europe/Paris dans /etc/timezone, crée le lien symbolique /etc/localtime
2. Lit /etc/timezone et vérifie que la valeur est bien Europe/Paris
3. Écrit Africa/Abidjan dans /etc/timezone, met à jour le lien symbolique
4. Lit /etc/timezone et vérifie que la valeur est bien Africa/Abidjan
5. Affiche le fuseau horaire final via debug

Play 4 – Load balancer

Configure nginx en reverse proxy avec upstream `least_conn` sur tous les serveurs web. Le load balancer écoute sur le **port 8081** pour coexister avec le serveur web sur le port 80 (les deux rôles tournent sur le même conteneur Docker dans cet environnement).



Play 5 – Nœud Windows (stub)

Limitation : Il n’a pas été possible de provisionner une VM Windows réelle. Quatre approches ont été tentées (émulation UTM x86, ISO ARM64 via uupdump, VM Azure cloud) et ont toutes échoué pour des raisons matérielles ou de quota cloud. Le détail complet est disponible dans `docs/07-windows-vm-attempts.md`.

Le workflow WinRM prévu est entièrement documenté dans `ansible/roles/windows_webserver/tasks/main.yml` et s’exécute sur un nœud Windows accessible avec les tâches suivantes : -
`community.windows.win_timezone` → Romance Standard Time (Europe/Paris) - Vérification via `win_shell: (Get-TimeZone).Id` -
`community.windows.win_timezone` → GMT Standard Time (Africa/Abidjan)

Notes de conception

Gestion des clés SSH

Les clés SSH sont stockées dans `~/.ansible-tp2/ansible_ed25519` (hors du workspace Git) afin que les opérations `git clean` de `actions/checkout` entre les jobs CI ne les suppriment pas. Le conteneur Docker est construit avec la clé publique intégrée dans `authorized_keys`. Le job Docker et le job Ansible référencent le même chemin stable via `$HOME/.ansible-tp2/`.

Conteneur Docker comme cible

La cible Ubuntu est un conteneur Docker (`ubuntu-target`) tournant sur la même machine que le runner auto-hébergé. Correspondance des ports :

Port hôte	Port conteneur	Usage
2222	22	SSH (Ansible)
8080	80	nginx webserver
8081	8081	nginx load balancer

Exécution en local

```
cd ansible
ansible-playbook -i inventory.ini site.yml

# Exclure le stub Windows
ansible-playbook -i inventory.ini site.yml --limit '!windows'

# Simulation (dry-run)
ansible-playbook -i inventory.ini site.yml --check
```

05 – Répartition de Charge avec nginx

Vue d'ensemble

nginx joue à la fois le rôle de **serveur web** (sur les hôtes webserver) et de **reverse proxy / load balancer** (sur l'hôte loadbalancer).

Client → :8081 (LB) → upstream { web1:80, web2:80, ... }

Note : Dans cet environnement Docker, le LB écoute sur le port 8081 (et non 80) pour coexister avec le serveur web sur le même conteneur.

Algorithme

least_conn — chaque nouvelle requête est transmise au backend ayant le moins de connexions actives. C'est plus adaptatif que le round-robin lorsque les requêtes ont des durées variables.

Configuration upstream générée (lb.conf.j2)

```
upstream backend {
    least_conn;
    server 192.168.1.10:80;    # serveur web 1
    server 192.168.1.11:80;    # serveur web 2
    # ... auto-généré par Ansible depuis groups['webserver']
}

server {
    listen 8081;

    location / {
        proxy_pass            http://backend;
        proxy_set_header      Host                $host;
        proxy_set_header      X-Real-IP           $remote_addr;
        proxy_set_header      X-Forwarded-For

```

```

$proxy_add_x_forwarded_for;
    proxy_connect_timeout 5s;
    proxy_read_timeout    30s;
}

location /health {
    return 200 "OK\n";    # endpoint de vérification de santé
}
}

```

La liste des serveurs upstream est **rendue dynamiquement** par Ansible — aucune édition manuelle n’est nécessaire lors de l’ajout ou de la suppression de nœuds.

Ajouter un nouveau serveur web

1. Ajouter son IP dans la variable GitHub Actions WEB_HOSTS (tableau JSON).
2. Pousser un commit sur main.
3. Terraform régénère l’inventaire → Ansible installe nginx sur le nouveau nœud **et** met à jour automatiquement le bloc upstream du LB.

Vérification de santé

```

curl http://127.0.0.1:8081/health
# → OK

```

Tester la distribution de charge

```

for i in $(seq 1 10); do curl -s http://127.0.0.1:8081/ | grep -o
'Server [0-9]*'; done

```

Les requêtes doivent être réparties entre les nœuds backend.

06 – Déroulement du Pipeline CI/CD

Déclencheur

Tout push sur la branche main déclenche automatiquement le pipeline.

Vue d’ensemble du pipeline

```

push sur main
├──
└── Job 1 : terraform (runner auto-hébergé) ───

```

```

1. checkout
2. setup terraform
3. terraform init
4. terraform validate
5. terraform apply → inventory.ini

```

```

Job 2 : docker (runner auto-hébergé)
1. checkout
2. gen-keys.sh → ~/.ansible-tp2/
3. docker compose up -d --build
4. attente SSH (15 tentatives × 3s)

```

```
needs: [terraform, docker]
```



```

Job 3 : ansible (runner auto-hébergé)
1. checkout
2. gen-keys.sh (clé déjà présente → skip)
3. écriture inventory.ini (heredoc shell)
   ansible_ssh_private_key_file=
     ~/.ansible-tp2/ansible_ed25519
4. vérification pip / ansible --version
5. ssh-keygen -R '[127.0.0.1]:2222'
6. ansible-playbook site.yml
   └─ play 1 : collecte de faits (localhost)
   └─ play 2 : déploiement nginx + HTML
   └─ play 3 : fuseau horaire Paris→Abidjan
   └─ play 4 : configuration load balancer
   └─ play 5 : stub Windows (injoignable)

```

Décisions clés du pipeline

Décision	Raison
runs-on: self-hosted	Garantit l'exécution sur le runner local, jamais sur un runner partagé
Clé SSH dans ~/.ansible-tp2/	Survit aux opérations git clean entre les jobs (les fichiers gitignores sont supprimés par checkout)
Inventaire écrit via heredoc	Pas de transfert d'artefact ; \$HOME s'étend correctement dans le chemin de la clé
ssh-keygen -R '[127.0.0.1]:2222'	Efface l'entrée known_hosts obsolète quand le conteneur est reconstruit
needs: [terraform, docker]	Impose l'exécution séquentielle (conteneur prêt avant Ansible)

Décision	Raison
<code>ignore_unreachable: true</code> sur le play Windows	Permet au pipeline de réussir même si la VM Windows n'est pas provisionnée

Résolution des problèmes

Permission denied (publickey) sur la connexion SSH Ansible

La clé privée SSH `~/.ansible-tp2/ansible_ed25519` doit : 1. Appartenir à l'utilisateur runner (User11), pas à root 2. Correspondre à la clé publique intégrée dans l'image Docker (`docker/ubuntu-target/authorized_keys`)

Si la propriété est incorrecte : `sudo chown -R User11:staff ~/.ansible-tp2`

Si la clé a changé (nouvelle clé, ancien conteneur) : relancer `docker/gen-keys.sh` puis `docker compose up -d --build` dans le répertoire `docker/`.

Le runner perd la communication avec le serveur

Il s'agit d'un problème réseau transitoire lors du nettoyage post-job de GitHub Actions. Relancer le job depuis l'interface Actions (Re-run failed jobs). Le playbook lui-même s'exécute correctement.

Preuve d'exécution sur le runner local

Dans l'interface GitHub Actions, l'en-tête du log de chaque job affiche le nom du runner. Il doit indiquer le nom du runner auto-hébergé (ex. `self-hosted`) — et non un runner hébergé par GitHub.

07 – Tentatives de Provisionnement de la VM Windows

Objectif

Le TP requiert un nœud Windows géré via WinRM pour l'automatisation du fuseau horaire. Ce document détaille chaque approche tentée et les raisons de l'échec sur ce matériel spécifique.

Contexte matériel

Propriété	Valeur
Machine	Apple Silicon Mac (ARM64 / puce M-series)
Espace disque libre	~11 Go
Hyperviseur utilisé	UTM (basé sur QEMU)

Tentative 1 — ISO Windows Server 2022 x86 via UTM (émulation)

Ce qui a été tenté

Téléchargement de l'ISO officielle **Windows Server 2022 Evaluation** (x86_64, build française) :

26100.32230.260111-0550.lt_release_svc_refresh_SERVER_EVAL_x64FRE_1
fr

Création d'une VM UTM et tentative de démarrage.

Raison de l'échec

Le Mac est **ARM64**. UTM peut émuler x86 via QEMU TCG, mais : - Le shell émulé n'avait **aucune commande fonctionnelle** — la traduction des instructions x86 était trop instable pour produire un environnement utilisable - Le démarrage était partiel : la VM tombait dans un shell CMD non fonctionnel, sans `ipconfig` ni `powershell`

Cause racine : L'émulation x86 sur Apple Silicon n'est pas adaptée à Windows Server. L'incompatibilité entre les jeux d'instructions rend l'OS inutilisable.

Tentative 2 — ISO Windows Server ARM64 native via uupdump.net

Ce qui a été tenté

Téléchargement d'un script de build ARM64 depuis **uupdump.net** :

26100.8521_arm64_en-us_core_886b936b_convert

Tentative de construction de l'ISO sur macOS avec le script `uup_download_macos.sh` fourni.

Raison de l'échec

Le script nécessite chntpw, qui dépend de openssl@1.0. Cette bibliothèque **ne compile pas sur Apple Silicon** :

```
Error: openssl@1.0 build fails on Apple Silicon (Mac M4)
EC point validation errors during tests
make[1]: *** [test_ec] Error 1
```

Il s'agit d'un problème connu en amont sans correctif disponible.

Contournement — Construction de l'ISO dans Docker

La chaîne d'outils macOS étant cassée, un conteneur Docker Linux (ARM64 Ubuntu) a été utilisé, qui dispose de packages chntpw fonctionnels :

```
docker run --rm \
-v "/Users/User11/Downloads/26100.8521_arm64_en-
us_core_886b936b_convert:/uup" \
ubuntu:22.04 bash -c "
  apt-get update -qq &&
  apt-get install -y cabextract wimtools chntpw genisoimage
  aria2 &&
  cd /uup && bash uup_download_linux.sh
"
```

L'ISO a été **construite avec succès** :

26100.1_CORE_ARM64_EN-US.ISO (3,9 Go)

Tentative 3 — Démarrage de l'ISO ARM64 dans UTM

Ce qui a été tenté

Création d'une nouvelle VM UTM (mode Virtualiser, pas Émuler) avec : - RAM : 2048 Mo | CPU : 2 cœurs | Disque : 15 Go - Réseau : Partagé (NAT) - ISO : 26100.1_CORE_ARM64_EN-US.ISO

Problème 1 — Shell EFI au lieu de l'installateur

La VM démarrait dans le **Shell Interactif UEFI v2.2** au lieu de lancer l'installateur.

Correctif : Lancement manuel du bootloader depuis le shell EFI :

```
FS0:
EFI/B00T/B00TAA64.EFI
```

Puis appui rapide sur une touche au message :

Press any key to boot from cd or dvd...

L'installateur Windows a ensuite démarré correctement.

Problème 2 — Espace disque insuffisant

Pendant l'installation, Windows Server a signalé :

| « Impossible d'installer sur ce lecteur — nécessite 52 Go ou plus »

Le disque VM était de 15 Go. L'augmenter à 60 Go était impossible car le Mac ne disposait que de **~11 Go d'espace libre**.

Cause racine : Espace disque épuisé — 11 Go libres sur l'hôte < 20 Go minimum requis pour l'installation de Windows Server Core.

Tentative 4 — VM Azure Cloud (Windows Server 2022 Datacenter)

Contexte

Après avoir épuisé les options locales, nous nous sommes tournés vers **Azure for Students** (crédit gratuit, abonnement 2e32bf2a-6410-4934-8182-fa5199a76f37, tenant efrei.net) pour provisionner une VM Windows cloud qu'Ansible pourrait atteindre via WinRM depuis le runner auto-hébergé.

Configuration réalisée

- Azure CLI installé via `pip3 install azure-cli`
- Authentification avec `az login --allow-no-subscriptions` (requis car l'abonnement étudiant n'apparaît pas lors du login standard)
- Groupe de ressources `devops-tp2-rg` créé avec succès dans `francecentral`

Problème 1 — Capacité Standard_B1s épuisée dans francecentral

```
(SkuNotAvailable) The requested VM size for resource  
'Following SKUs have failed for Capacity Restrictions:  
Standard_B1s'  
is currently not available in location 'FranceCentral'.
```

Standard_B2s produisait la même erreur. La série B est la seule famille avec un quota non nul sur Azure for Students, et la capacité physique Azure dans `francecentral` était épuisée au moment de la tentative.

Problème 2 — La politique Azure restreint le déploiement à francecentral

Tentatives dans d'autres régions (northeurope, eastus, westus2, westeurope, uksouth, australiaeast, japaneast) :

(RequestDisallowedByAzure) Resource was disallowed by Azure:
This policy maintains a set of best available regions where your subscription can deploy resources.

Seules francecentral, australiaeast et southeastasia acceptaient la création de groupes de ressources. La création de VM dans australiaeast et southeastasia retournait également disallowed pour les ressources compute.

Problème 3 — Quota = 0 pour toutes les familles non-B

Test d'autres tailles de VM dans francecentral :

(QuotaExceeded) Current Limit: 0, Current Usage: 0, Additional Required: 2

Toutes les familles DSv2, DSv3, DSv4, DSv5 et autres ont un quota fixe de **0 vCores** sur l'abonnement Azure for Students. Seule la série B disposait d'un quota supérieur à 0, mais aucune capacité physique n'était disponible.

Résultat

Le provisionnement d'une VM Azure est impossible sur cet abonnement dans toute région accessible :

Famille	francecentral	australiaeast	southeastasia
Standard_B*	SkuNotAvailable (capacité)	Interdit	Interdit
Standard_DS*	QuotaExceeded (limite=0)	Interdit	Interdit
Standard_D*v3+	QuotaExceeded (limite=0)	Interdit	Interdit

Il s'agit d'une contrainte connue des abonnements gratuits Azure for Students — le quota compute est minimal et la capacité dans les régions européennes est fortement sollicitée.

Conclusion

Tentative	Bloquant
Émulation x86 ISO sur ARM	Incompatibilité CPU — shell inutilisable
Chaîne d'outils ISO macOS	openssl@1.0 ne compile pas sur Apple Silicon
ISO ARM construite via Docker	Succès — ISO construite correctement

Tentative	Bloquant
Démarrage ISO ARM dans UTM	Contournement EFI nécessaire mais fonctionnel
Installation Windows	Espace disque épuisé (11 Go libres vs 52 Go requis)
VM Azure cloud	Quota 0 vCores + capacité B-series épuisée dans toutes les régions accessibles

Ce qu’aurait fait l’automatisation Windows

Si la VM avait été provisionnée, le play Ansible aurait :

1. Connecté via **WinRM HTTP** (port 5985, authentification NTLM)
2. Exécuté `community.windows.win_timezone` pour définir Romance Standard Time (Europe/Paris)
3. Vérifié via `win_shell: (Get-TimeZone).Id`
4. Basculé sur GMT Standard Time (Africa/Abidjan)
5. Vérifié à nouveau

Le code complet est conservé dans `ansible/roles/windows_webserver/tasks/main.yml`.

Commandes WinRM de configuration (pour référence)

```
winrm quickconfig -q
winrm set winrm/config/service/auth '@{Basic="true"}'
winrm set winrm/config/service '@{AllowUnencrypted="true"}'
netsh advfirewall firewall add rule name="WinRM HTTP"
        protocol=TCP dir=in localport=5985 action=allow
```